

Universidad de San Carlos de Guatemala

Division de las Ciencias de la Ingenieria

Ingenieria en Sistemas

Organizacion De Lenguajes Y Compiladores 1

Seccion: A

Auxiliar: Carlos Fernando Enrique Lopez Garcia



“Manual de Usuario Proyecto Golite Fase 2”

Estudiante:

Mynor Miguel Monzón Martínez 3195097021306

Guatemala, 27 de junio de

2026

Introduccion

Este manual describe el uso del Interprete GoLite, una aplicacion de escritorio desarrollada en Java que permite escribir, editar y ejecutar codigo en el lenguaje GoLite directamente desde una interfaz grafica. GoLite es un subconjunto del lenguaje Go disenado con fines educativos para el curso de Organizacion de Lenguajes y Compiladores 1.

La Fase 2 extiende las capacidades del interprete incorporando nuevas construcciones del lenguaje: slices unidimensionales y multidimensionales, structs con metodos, funciones con parametros y retorno, sentencia switch-case y bucle for range. Adicionalmente se incorporan dos nuevos reportes: la Tabla de Simbolos y el Reporte AST visual generado con Graphviz.

El sistema permite abrir y guardar archivos de codigo con extension .glt, ejecutar el interprete sobre el codigo escrito, visualizar la salida en una consola integrada, y consultar los cuatro reportes disponibles en el menu Reportes.

1. Requisitos del Sistema

Para ejecutar el Interprete GoLite se necesita lo siguiente:

- Sistema operativo: Ubuntu Linux (recomendado), o cualquier sistema con soporte para Java.
- Java Runtime Environment (JRE) version 17 o superior.
- Graphviz instalado en el sistema para la generacion del reporte AST como imagen PNG. En Ubuntu: `sudo apt install graphviz`
- No se requiere instalacion adicional de software. El programa se distribuye como un archivo JAR ejecutable que incluye todas sus dependencias.

2. Instalacion y Ejecucion

2.1 Obtener el programa

El programa se distribuye como un archivo JAR ejecutable. Se puede obtener de dos formas:

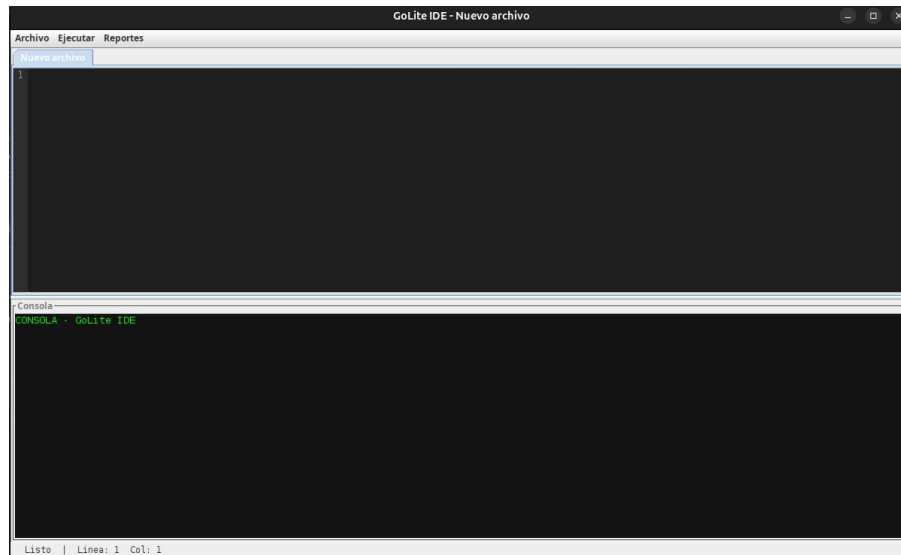
- Descargando el Release publicado en el repositorio de GitHub del proyecto. El JAR se encuentra como asset adjunto al release.
- Compilando el codigo fuente con Maven desde la carpeta GoLiteInterpreter/ del repositorio con el comando `mvn clean package`.

2.2 Ejecutar el programa

Para iniciar el interprete, abrir una terminal en la carpeta donde se encuentra el archivo JAR y ejecutar el siguiente comando:

```
java -jar GoLiteInterpreter-1.0-SNAPSHOT.jar
```

Al ejecutarlo, la ventana principal del interprete se abrira automaticamente.



3. Interfaz Principal

La ventana principal del interprete esta dividida en las siguientes areas:

Area	Descripcion
Editor de codigo	Area principal donde se escribe o pega el codigo GoLite con fuente monoespaciada. Muestra la linea y columna actual del cursor. Soporta multiples archivos abiertos en tabs.
Consola de salida	Panel inferior donde aparece la salida generada por <code>fmt.Println</code> y los mensajes del interprete.
Barra de menu	Contiene los menus Archivo (nuevo, abrir, guardar), Ejecutar y Reportes.
Reportes	Menu con cuatro opciones: Tabla de Tokens, Reporte de Errores, Tabla de Simbolos y Reporte AST.

4. Funcionalidades del Sistema

4.1 Crear un archivo nuevo

Para limpiar el editor y comenzar con un archivo en blanco, ir al menu Archivo y seleccionar Nuevo. Se abrira una nueva pestana vacia en el editor.

4.2 Abrir un archivo .glt

Para abrir un archivo de codigo GoLite existente, ir al menu Archivo y seleccionar Abrir. Se abrira un explorador de archivos donde se debe navegar hasta el archivo con extension .glt y seleccionarlo. El contenido del archivo se cargara en una nueva pestana del editor automaticamente.

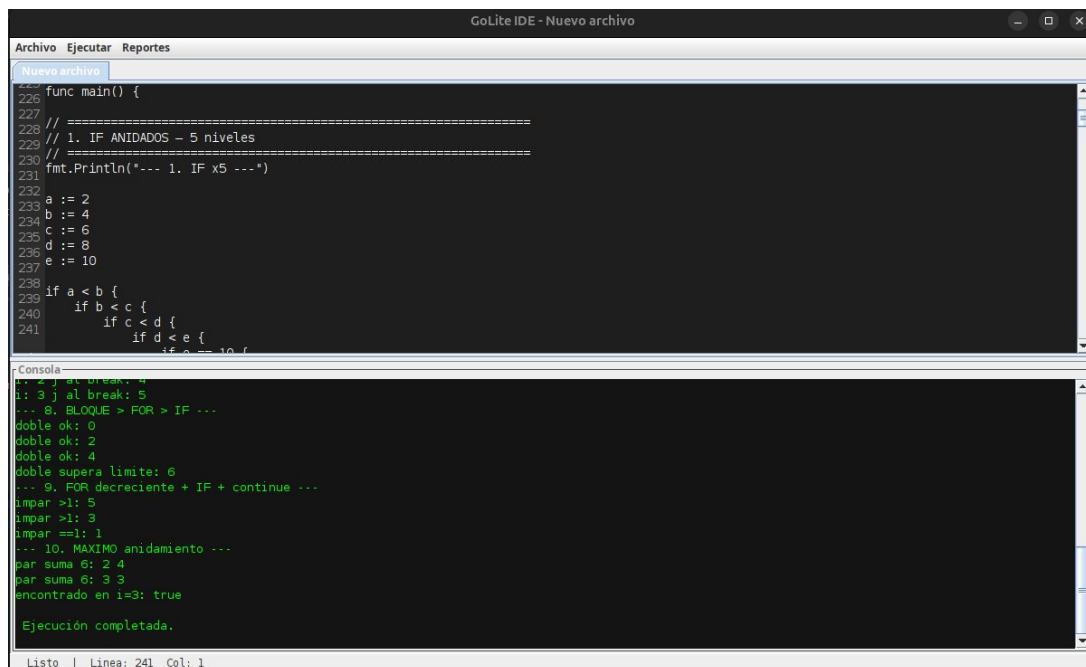
4.3 Guardar un archivo .glt

Para guardar el codigo actual, ir al menu Archivo y seleccionar Guardar. Si el archivo ya tiene nombre, se guardara en la misma ubicacion. Si es un archivo nuevo, se abrira un dialogo para elegir la ubicacion y el nombre. El archivo se guardara con la extension .glt.

4.4 Ejecutar el codigo

Para interpretar el codigo escrito en el editor, ir al menu Ejecutar y seleccionar Ejecutar. El interprete realizara el analisis lexico, sintactico, semantico y ejecutara las instrucciones del programa.

La salida producida por las instrucciones `fmt.Println` aparecera en la consola de salida. Si se detectan errores lexicos, estos se reportan pero la ejecucion continua. Si se detecta un error semantico, la ejecucion se detiene en ese punto.



The screenshot displays the GoLite IDE interface. The top menu bar includes 'Archivo', 'Ejecutar', and 'Reportes'. The main editor window, titled 'Nuevo archivo', contains the following Go code:

```
func main() {  
    // =====  
    // 1. IF ANIDADOS - 5 niveles  
    // =====  
    fmt.Println("--- 1. IF x5 ---")  
    a := 2  
    b := 4  
    c := 6  
    d := 8  
    e := 10  
    if a < b {  
        if b < c {  
            if c < d {  
                if d < e {  
                    if e == 10 {  
                        fmt.Println("MAXIMO anidamiento")  
                    }  
                }  
            }  
        }  
    }  
    for i := 0; i < 10; i++ {  
        if i % 2 == 0 {  
            fmt.Println("doble ok:", i)  
        } else {  
            fmt.Println("impar >1:", i)  
        }  
        if i == 3 {  
            break  
        }  
    }  
    for i := 10; i > 0; i-- {  
        if i % 2 == 0 {  
            fmt.Println("doble supera limite:", i)  
        } else {  
            fmt.Println("impar >1:", i)  
        }  
        if i == 3 {  
            continue  
        }  
    }  
    encontrado := false  
    for i := 0; i < 10; i++ {  
        if i == 3 {  
            encontrado = true  
            break  
        }  
    }  
    fmt.Println("encontrado en i=3: ", encontrado)  
}
```

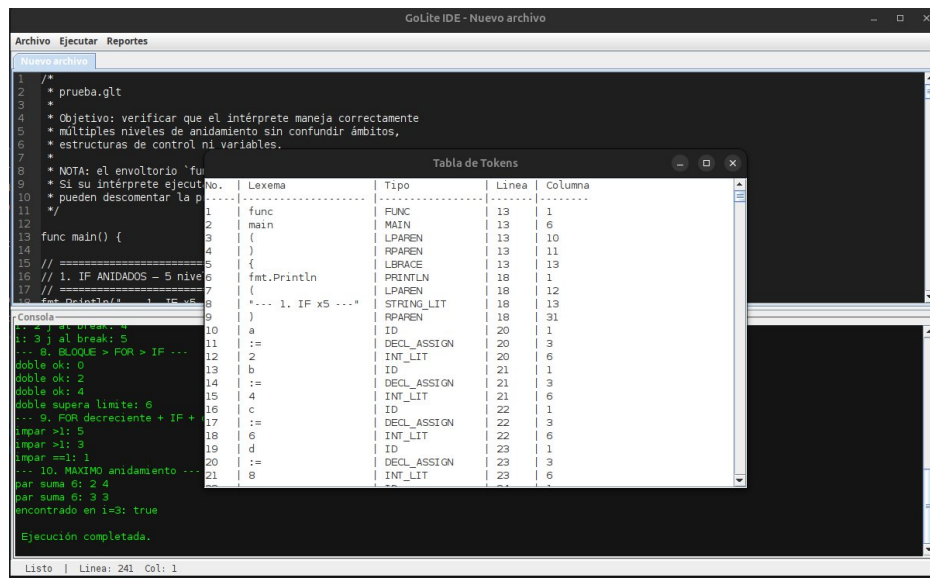
The bottom console window shows the execution output:

```
--- 1. IF x5 ---  
doble ok: 0  
doble ok: 2  
doble ok: 4  
doble supera limite: 6  
--- 9. FOR decreciente + IF + continue ---  
impar >1: 5  
impar >1: 3  
impar ==1: 1  
--- 10. MAXIMO anidamiento ---  
par suma 6: 2 4  
par suma 6: 3 3  
encontrado en i=3: true  
Ejecución completada.
```

The status bar at the bottom indicates 'Listo | Linea: 241 Col: 1'.

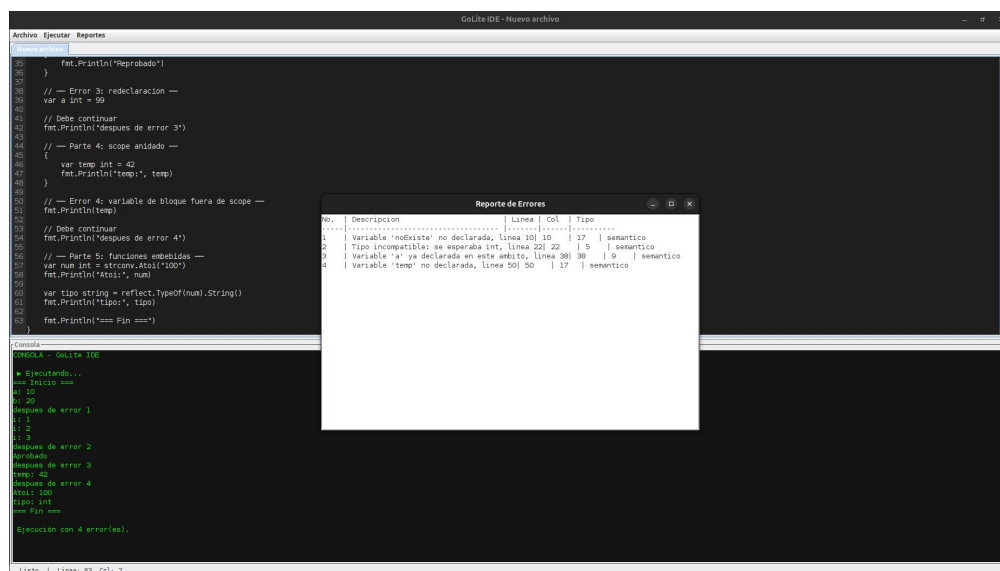
4.5 Ver el reporte de tokens

Después de ejecutar el código, ir al menú Reportes y seleccionar Tabla de Tokens. Se abrirá una ventana con todos los tokens reconocidos durante el análisis léxico, mostrando el número, lexema, tipo, línea y columna de cada token.



4.6 Ver el reporte de errores

Si durante el análisis se detectan errores léxicos, sintácticos o semánticos, ir al menú Reportes y seleccionar Reporte de Errores. Cada error indica su número, descripción del problema, línea, columna y tipo de error.



4.7 Ver la Tabla de Simbolos

Despues de ejecutar el codigo, ir al menu Reportes y seleccionar Tabla de Simbolos. Se abrira una ventana con todos los simbolos declarados en el programa, incluyendo structs, funciones y variables, con su tipo, ambito, linea y columna de declaracion.

4.8 Ver el Reporte AST

Despues de ejecutar el codigo, ir al menu Reportes y seleccionar Reporte AST. El sistema generara automaticamente el arbol de sintaxis abstracta del programa en formato visual. La ventana muestra dos pestanas:

- Imagen AST: el arbol visual renderizado como imagen PNG con scroll para navegar.
- Codigo DOT: el texto en formato Graphviz para copiar a herramientas online como <https://dreampuf.github.io/GraphvizOnline>

En la parte inferior aparece la ruta donde se guardo el archivo PNG, el boton Copiar DOT para copiar el codigo al portapapeles, y el boton Abrir carpeta reportes para ver los archivos generados.

Nota: Para que se genere la imagen PNG es necesario tener Graphviz instalado en el sistema. Si no esta instalado, solo se mostrara la pestana Codigo DOT.

5. Referencia del Lenguaje GoLite

Esta seccion describe las construcciones del lenguaje GoLite que el interprete puede ejecutar en la Fase 2.

5.1 Tipos de datos

Tipo	Descripcion	Valor defecto	por	Ejemplo
int	Numero entero	0		var x int = 10
float64	Numero decimal de 64 bits	0.0		var y float64 = 3.14
string	Cadena de caracteres	""		var s string = "hola"
bool	Valor logico verdadero o falso	false		var b bool = true
rune	Caracter Unicode	0		var c rune = 'A'
[]int	Slice de enteros	nil		nums := []int{1, 2, 3}

Tipo	Descripcion	Valor por defecto	Ejemplo
[]float64	Slice de flotantes	nil	fs := []float64{1.0, 2.0}
[]string	Slice de cadenas	nil	ss := []string{"a", "b"}
[][]int	Slice bidimensional	nil	mtx := [][]int{{1,2},{3,4}}
struct	Tipo compuesto definido por el usuario	nil	Persona p = {nombre: "Ana"}

5.2 Declaracion de variables

```

var nombre int = 10           // tipo explicito con valor
var nombre int                // tipo explicito, valor por defecto
nombre := 10                  // inferencia de tipo
var s []int                    // slice vacio, valor nil

```

5.3 Operadores disponibles

Categoria	Operadores	Ejemplo
Aritmeticos	+ - * / %	10 + 5 * 2 -> 20
Comparacion	== != < > <= >=	10 > 5 -> true
Logicos	&& !	true && false -> false
Asignacion compuesta	+= -= *= /=	x *= 2 (equivale a x = x * 2)
Incremento / decremento	++ --	i++ (equivale a i += 1)

5.4 Sentencia if-else

```

if condicion {
    // instrucciones
} else if otraCondicion {
    // instrucciones
} else {
    // instrucciones
}

```

5.5 Sentencia switch-case

```
switch expresion {  
    case valor1:  
        // instrucciones  
    case valor2:  
        // instrucciones  
    default:  
        // instrucciones  
}
```

5.6 Sentencia for

Forma tipo while:

```
for condicion {  
}
```

Forma clasica con inicializacion, condicion e incremento:

```
for i := 0; i < 10; i++ {  
}
```

Forma for range sobre un slice:

```
for indice, valor := range miSlice {  
}  
for _, valor := range miSlice { // ignorar indice  
}
```

5.7 Slices

Creacion e inicializacion:

```
nums := []int{1, 2, 3, 4, 5}  
var vacio []int // valor nil  
vacio = append(vacio, 10) // agrega elemento
```

Operaciones disponibles:

Funcion	Descripcion	Ejemplo
append(s, val)	Agrega un elemento al final del slice y retorna el nuevo slice	s = append(s, 99)
len(s)	Retorna la longitud del slice	fmt.Println(len(s))
slices.Index(s, val)	Retorna el indice de la primera	idx := slices.Index(s,

Funcion	Descripcion	Ejemplo
	coincidencia, -1 si no existe	30)
strings.Join(s, sep)	Une elementos de un []string con un separador	strings.Join(s, ", ")
s[i]	Acceso al elemento en la posicion i	fmt.Println(s[0])
s[i] = val	Asignacion a la posicion i	s[0] = 99

Slices multidimensionales:

```

mtx := [][]int{
    {1, 2, 3},
    {4, 5, 6},
}
fmt.Println(mtx[0][1])    // 2
mtx[1][2] = 99

```

5.8 Structs

Definicion (solo en ambito global):

```

struct Persona {
    string nombre;
    int edad;
}

```

Instanciacion (dos sintaxis aceptadas):

```

// Con nombre del tipo en el lado derecho
Persona p = Persona{nombre: "Alice", edad: 25}

// Sin nombre del tipo (sintaxis simplificada)
Persona p = {nombre: "Alice", edad: 25}

```

Acceso y modificacion de atributos:

```

fmt.Println(p.nombre)
p.edad = 30

```

Structs anidados:

```

fmt.Println(p.direccion.calle)
p.direccion.numero = 100

```

5.9 Funciones

Declaracion con parametros y retorno:

```
func sumar(a int, b int) int {  
    return a + b  
}
```

Funcion sin retorno (void):

```
func saludar(nombre string) {  
    fmt.Println("Hola,", nombre)  
}
```

Metodo de struct:

```
func (p Persona) presentarse() {  
    fmt.Println("Soy", p.nombre)  
}  
p.presentarse()
```

5.10 Funciones embebidas

Funcion	Descripcion	Ejemplo
fmt.Println(...)	Imprime uno o mas valores separados por espacio con salto de linea	fmt.Println("Hola", 42)
strconv.Atoi(s)	Convierte una cadena a entero	n := strconv.Atoi("123")
strconv.ParseFloat(s)	Convierte una cadena a float64	f := strconv.ParseFloat("3.14")
reflect.TypeOf(x)	Retorna el tipo de la expresion como cadena	reflect.TypeOf(42) -> int
append(s, val)	Agrega un elemento al slice y retorna el nuevo slice	s = append(s, 5)
len(s)	Retorna la longitud de un slice	len([]int{1,2,3}) -> 3
slices.Index(s, val)	Busca un valor en el slice y retorna su indice	slices.Index(s, 30) -> 2
strings.Join(s, sep)	Une los elementos de un []string con un separador	strings.Join(s, "-")

6. Ejemplo Completo de Uso

A continuacion se muestra un ejemplo de sesion completa usando el interprete con funcionalidades de Fase 2.

Paso 1: Abrir el programa

Ejecutar el JAR como se indico en la seccion 2.2. La ventana principal aparecera en pantalla.

Paso 2: Escribir el codigo

En el editor, escribir el siguiente programa de ejemplo:

```
struct Producto {
    string nombre;
    float64 precio;
}

func (prod Producto) mostrar() {
    fmt.Println(prod.nombre, "->", prod.precio)
}

func aplicarDescuento(productos []float64, pct float64) []float64 {
    resultado := []float64{}
    for _, p := range productos {
        resultado = append(resultado, p - (p * pct / 100.0))
    }
    return resultado
}

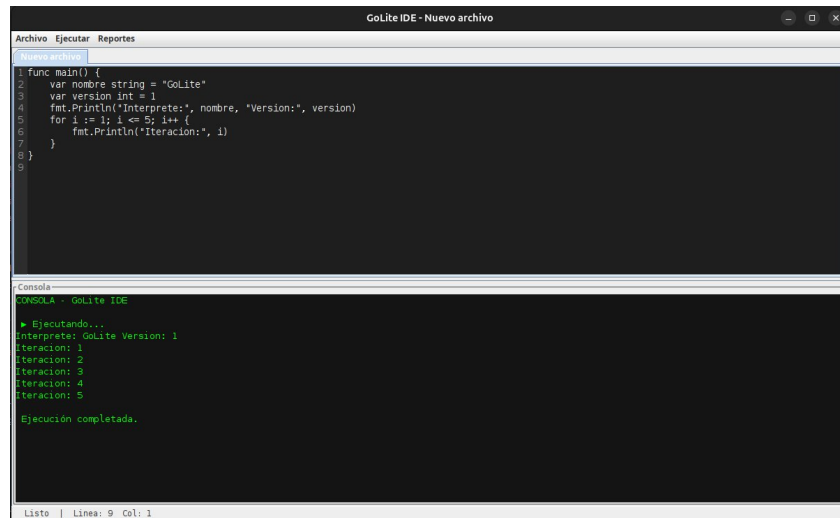
func main() {
    Producto p1 = {nombre: "Laptop", precio: 999.99}
    Producto p2 = {nombre: "Mouse", precio: 25.50}
    p1.mostrar()
    p2.mostrar()

    precios := []float64{999.99, 25.50, 150.0}
    nuevos := aplicarDescuento(precios, 10.0)
    fmt.Println(nuevos)
    fmt.Println(len(nuevos))
}
```

Paso 3: Ejecutar

Ir al menu Ejecutar y seleccionar Ejecutar. La consola de salida mostrara:

```
Laptop -> 999.99  
Mouse -> 25.5  
[899.991 22.95 135.0]  
3
```



The screenshot shows the GoLite IDE window titled "GoLite IDE - Nuevo archivo". It has three tabs: "Archivo", "Ejecutar", and "Reportes". The "Nuevo archivo" tab is active, displaying a Go program:

```
1 func main() {  
2     var nombre string = "GoLite"  
3     var version int = 1  
4     fmt.Println("Interprete:", nombre, "Version:", version)  
5     for i := 1; i <= 5; i++ {  
6         fmt.Println("Iteracion:", i)  
7     }  
8 }  
9
```

Below the code editor is the "Consola" panel, which shows the execution output:

```
CONSOLE - GoLite IDE  
• Ejecutando...  
Interprete: GoLite Version: 1  
Iteracion: 1  
Iteracion: 2  
Iteracion: 3  
Iteracion: 4  
Iteracion: 5  
Ejecución completada.
```

At the bottom of the console, it says "Listo | Línea: 9 | Col: 1".

Paso 4: Revisar la Tabla de Simbolos

Ir al menu Reportes y seleccionar Tabla de Simbolos. Se mostrara una ventana con todos los simbolos declarados: el struct Producto, las funciones mostrar y aplicarDescuento, y las variables del main.

Paso 5: Ver el Reporte AST

Ir al menu Reportes y seleccionar Reporte AST. Se abra la ventana con el arbol visual del programa en la pestana Imagen AST.

Paso 6: Guardar el archivo

Ir al menu Archivo y seleccionar Guardar. Asignar el nombre deseado al archivo y guardarlo con extension .glt.

7. Mensajes de Error y Solucion

Tipo	Descripcion del error	Causa	Solucion
Lexico	Caracter no reconocido: '@'	Se uso un caracter que no pertenece al lenguaje.	Revisar el codigo y eliminar el caracter no valido.
Sintactico	Error sintactico cerca de 'desconocido'	El bloque tiene un token inesperado o le falta uno.	Verificar que todos los bloques esten bien delimitados con { }.
Semantico	Variable 'x' no declarada en este ambito	Se uso una variable antes de declararla.	Declarar la variable antes de usarla con var o :=.
Semantico	No se puede asignar float64 a variable de tipo int	Se intento asignar un valor de tipo incompatible.	Usar el tipo correcto o declarar la variable como float64.
Semantico	Division entre cero	El divisor de una operacion es cero.	Verificar que el denominador no sea cero antes de dividir.
Semantico	Operacion invalida sobre nil	Se aplico una operacion sobre un slice con valor nil.	Inicializar el slice antes de usarlo: s = append(s, val).
Semantico	Funcion 'X' ya declarada	Se declaro una funcion con un nombre que ya existe.	Usar nombres unicos para cada funcion.
Semantico	Struct 'X' ya declarado	Se declaro un struct con un nombre que ya existe.	Usar nombres unicos para cada struct.
Semantico	Indice fuera de rango	Se intento acceder a una posicion que no existe en el slice.	Verificar que el indice sea menor que len(slice).

8. Archivos Soportados

- El editor abre y guarda archivos con extension .glt (GoLite source file).
- El archivo debe estar codificado en UTF-8.
- Los reportes AST se guardan automaticamente en la carpeta reportes/ con el mismo nombre del archivo .glt ejecutado, en formatos .dot y .png.
- No hay limite de tamaño de archivo impuesto por el interprete.

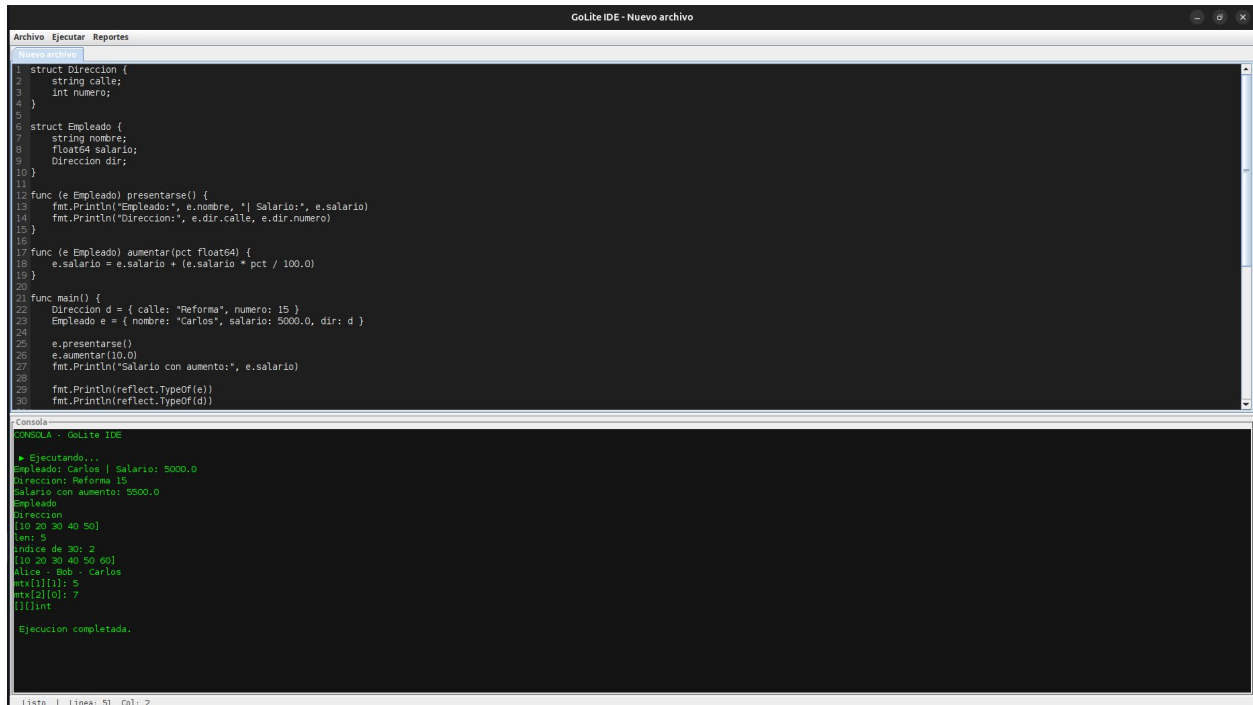
9. Notas Adicionales

- El lenguaje GoLite es sensible a mayusculas y minusculas. La palabra if no es lo mismo que IF.
- Las llaves { } son obligatorias en todas las estructuras de control.
- El punto y coma al final de cada sentencia es opcional.
- Los comentarios de una sola linea se escriben con // y los de varias lineas con /* ... */.
- Una variable declarada en un bloque interno oculta a una variable con el mismo nombre en un bloque externo y deja de existir al terminar ese bloque.
- Los tipos no pueden cambiar una vez declarada la variable.
- Los structs solo pueden declararse en el ambito global, fuera de cualquier funcion.
- Los slices declarados con var sin valor inicial tienen valor nil. Operaciones como len() o acceso por indice sobre nil generan error semantico. Se debe inicializar con append o con un literal antes de usar.
- El operador append siempre retorna el nuevo slice y debe reasignarse: s = append(s, val).

Fase 2 — Capturas de Funcionamiento

A continuacion se presentan capturas del interprete GoLite Fase 2 ejecutando programas que demuestran las nuevas funcionalidades implementadas.

Ejecucion de programa con structs y slices



The screenshot shows the GoLite IDE interface. The top window displays a Go program with structs and functions. The bottom window shows the execution output in the console.

```
1 struct Direccion {
2     string calle;
3     int numero;
4 }
5
6 struct Empleado {
7     string nombre;
8     float64 salario;
9     Direccion dir;
10 }
11
12 func (e Empleado) presentarse() {
13     fmt.Println("Empleado:", e.nombre, "Salario:", e.salario)
14     fmt.Println("Direccion:", e.dir.calle, e.dir.numero)
15 }
16
17 func (e Empleado) aumentar(pct float64) {
18     e.salario = e.salario + (e.salario * pct / 100.0)
19 }
20
21 func main() {
22     Direccion d = { calle: "Reforma", numero: 15 }
23     Empleado e = { nombre: "Carlos", salario: 5000.0, dir: d }
24
25     e.presentarse()
26     e.aumentar(10.0)
27     fmt.Println("Salario con aumento:", e.salario)
28
29     fmt.Println(reflect.TypeOf(e))
30     fmt.Println(reflect.TypeOf(d))
31 }
```

Console Output:

```
GOING - GoLite IDE
# Ejecutando...
Empleado: Carlos | Salario: 5000.0
Direccion: Reforma 15
Salario con aumento: 5500.0
Empleado
Direccion
[10 20 30 40 50]
len: 5
Indice de 30: 2
[10 20 30 40 50 60]
Alice - Bob - Carlos
mtx[1][1]: 5
mtx[2][0]: 7
[]int
Ejecucion completada.
```

Reporte de Tabla de Simbolos

El reporte muestra todos los simbolos declarados en el programa: structs globales, funciones y variables, con su tipo, ambito, linea y columna de declaracion.

Tabla de Simbolos						
No.	ID	Tipo Simbolo	Tipo Dato	Ambito	Linea	Co
1	Empleado	Struct	Empleado	Global	6	8
2	Direccion	Struct	Direccion	Global	1	8
3	mtx	Variable	auto	Global	43	5
4	numeros	Variable	auto	Global	32	5
5	d	Variable	Direccion	Global	22	15
6	e	Variable	Empleado	Global	23	14
7	nombres	Variable	auto	Global	40	5

Reporte AST — Imagen generada por Graphviz

El reporte AST genera automaticamente el arbol de sintaxis abstracta del programa en formato visual. La imagen se guarda en la carpeta reportes/ con el nombre del archivo ejecutado.

